

# StabiFormer-RR: Artifact-Invariant Time–Frequency Optimal-Transport Learning for RR-Interval Rehabilitation Monitoring in Disability-Oriented Settings

Detailed Technical Report + Implementation Blueprint

February 2026

## Abstract

This report proposes **StabiFormer-RR**, a purpose-built deep learning framework for extracting **artifact-invariant, physiologically meaningful** representations from **ECG-derived RR-interval (interbeat) sequences** and converting them into **objective rehabilitation monitoring markers** for disability-oriented care. The core contribution is a new **band-limited time–frequency optimal-transport loss** operating on a differentiable time–frequency representation of RR-derived dynamics (e.g., instantaneous heart rate / tachogram), explicitly focusing optimization on physiologically relevant HRV frequency ranges while preventing transient artifacts (missed beats, false detections, motion-induced distortions) from dominating learning. A complementary **stability-consistency objective** enforces invariance of the learned marker under realistic RR corruption processes that emulate rehabilitation motion and sensor instability. We provide clean mathematical formulations, an architecture blueprint, training pseudocode, a minimal PyTorch scaffold, a CVPR-shaped ablation plan, and a future-work pathway toward EEG-based cortical stability and multimodal EEG–ECG coupling.

## 1 Proposed Topic

**Proposed Topic:** *Artifact-invariant learning of physiologically constrained time–frequency representations from ECG-derived RR-interval sequences to derive objective rehabilitation progress markers for disability-oriented monitoring.*

## 2 Title

**StabiFormer-RR: Artifact-Invariant Time–Frequency Optimal-Transport Learning for RR-Interval Rehabilitation Monitoring**

## 3 One-line Idea

Train a lightweight model to produce a physiologically meaningful **rehabilitation stability marker** from **RR-interval sequences** using (i) a **band-limited time–frequency optimal-transport (OT) loss** and (ii) **artifact-consistency regularization** under realistic RR corruptions; then compute a session-level **progress index** (stability improves over time/sessions).

## 4 Why this is directly related to Disabilities (clean argument)

Neurological disabilities (e.g., stroke, spinal cord injury, Parkinsonian motor disability) often require long-term rehabilitation, where **autonomic regulation and physiological stability** are clinically meaningful but difficult to measure reliably during therapy due to movement and sensor noise. RR-derived autonomic dynamics (HRV) can serve as a practical substrate for continuous monitoring, but real rehabilitation conditions introduce beat detection errors and nonstationary artifacts. StabiFormer-RR targets this deployment gap by learning an artifact-invariant stability marker from RR dynamics intended to support **disability rehabilitation monitoring** (not diagnosis).

## 5 The Real Gap (What is missing)

### 5.1 G1: HRV pipelines depend on fragile pre-processing under motion

Classical HRV analysis assumes reasonably accurate R-peak detection and stable recordings. In rehabilitation settings, motion can cause missed beats and false detections, producing RR sequences with spikes and dropouts that bias downstream HRV features. A learning framework that is *explicitly robust* to these RR-level artifacts is missing.

### 5.2 G2: Training objectives are misaligned with physiological meaning under RR artifacts

Many deep approaches optimize time-domain losses (MAE/MSE) on signals or features. For RR sequences, artifacts create large local deviations that dominate MAE/MSE, encouraging models to fit artifact energy rather than physiologically meaningful autonomic dynamics.

### 5.3 G3: Robustness is rarely designed as an explicit invariance objective

Most papers *evaluate* robustness but do not *train* for invariance: “same physiology, different artifacts” should yield the same stability marker. Rehabilitation monitoring needs invariance-by-design, not hope-by-generalization.

### 5.4 G4: Existing time–frequency losses are not distributional and not physiology-band constrained

Standard spectral penalties (e.g., L2 on PSD) are pointwise and can be overly sensitive to corruption. A distributional objective (OT/Wasserstein-type) that matches time–frequency *distributions* inside HRV-relevant bands is largely absent.

## 6 The Novelty (What you add)

### 6.1 N1: Band-Limited Time–Frequency Optimal Transport (BL-TFOT) on RR-derived dynamics

We build a differentiable time–frequency map from RR dynamics (e.g., interpolated tachogram or instantaneous HR) and match predicted vs. target *distributions* using entropic OT, restricted to physiologically relevant HRV frequency ranges (band-limited). This reduces domination by transient RR artifacts while preserving meaningful autonomic dynamics.

## 6.2 N2: RR Artifact Consistency Regularization (SCR-RR)

We enforce invariance of the marker under realistic RR corruptions that emulate rehabilitation motion and detection instability:

- **Missed beats** (RR doubling / local dropouts)
- **False detections** (short RR insertions)
- **Jitter** (small timing noise)
- **Burst noise** (clusters of errors)

This is not generic augmentation; it is **mechanism-aligned** with RR extraction failure modes.

## 6.3 N3: Purpose-built efficient architecture (StabiFormer-RR)

StabiFormer-RR is optimized for RR sequence modeling:

- **Local Conv stem** to capture short-range RR patterns.
- **Long-context temporal block** (efficient, trend-aware) to model autonomic regulation over longer windows.
- **Tiny attention mixer** for lightweight global mixing (optional for 1D RR).
- **Multi-head output**: (i) reconstructed RR/HR dynamics (optional), (ii) latent stability embedding, (iii) scalar progress marker.

## 6.4 N4: Artifact-Invariance Benchmark Protocol for RR

We introduce a standardized RR corruption suite and report:

- **Marker invariance error** under corruption (lower is better)
- **Band fidelity** in HRV-relevant bands
- **Progress stability** (window/session stability index)

# 7 Datasets (so reviewers don't reject)

We adopt a reviewer-safe strategy: **method validation on open PhysioNet resources** with RR/beat annotations, plus an explicit statement that disability-cohort clinical validation is future work.

### 7.1 Primary datasets (RR / beat-annotated ECG)

**MIT-BIH Arrhythmia Database (PhysioNet).** The reference annotation files place beat labels near the R-wave peak and are described as usable for HRV studies requiring accurate beat timing [1, 2]. This enables robust RR extraction and controlled corruption injection.

**Congestive Heart Failure RR Interval Database (PhysioNet).** Provides RR interval time series for long-term recordings, supporting evaluation of long-range autonomic dynamics and marker stability [3].

## 7.2 Supporting resources (reproducible HRV computation)

**PhysioNet Cardiovascular Signal Toolbox / HRV resources.** Provides evidence-based HRV computation utilities and standard metrics, useful for baseline comparisons and reproducibility [4].

## 7.3 Why these datasets prevent rejection

- They are **public, widely used, reproducible** PhysioNet resources.
- They provide **beat timing/annotations or RR series** suitable for RR modeling and HRV evaluation [1, 3].
- They allow a clean methodological paper: invariance, robustness, time–frequency fidelity.

## 7.4 Positioning to address “no disability rehab cohort”

We explicitly state:

*This study introduces an artifact-invariant RR-sequence learning framework and validates robustness and physiological fidelity on large-scale open ECG/RR datasets. Clinical validation on disability-specific rehabilitation cohorts is future work.*

# 8 Implementation Blueprint (step-by-step)

## 8.1 Step 0: Define RR representation and frequency bands

From ECG (with annotations or R-peak detector), compute RR intervals:

$$r_k = t_k - t_{k-1}.$$

Convert to a uniformly sampled series for time–frequency analysis (tachogram / IHR):

$$\text{IHR}(t) = \frac{60}{r(t)} \quad (\text{bpm}), \quad r(t) \text{ obtained by interpolation of } \{(t_k, r_k)\}.$$

Define HRV-relevant band mask  $B(f)$  (example bands; adapt per protocol):

- **LF/HF-style monitoring:** focus on low frequencies up to  $\sim 0.4$  Hz (common HRV range).
- **Task/rehab sessions:** optionally extend or restrict bands depending on session duration and sampling.

## 8.2 Step 1: RR corruption generator (rehabilitation-realistic)

Create corrupted RR views  $\tilde{r}^{(v)}$  from a clean RR series:

- Missed beat: replace  $r_k$  with  $r_k + r_{k+1}$  (merge) + remove one beat
- False detection: split  $r_k$  into two short intervals (insert beat)
- Jitter:  $r_k \leftarrow r_k + \eta$ , small  $\eta$
- Dropout: mask a segment and interpolate (simulate sensor loss)

### 8.3 Step 2: Differentiable time–frequency representation

Compute STFT magnitude on uniformly sampled IHR(t) (or tachogram):

$$S_x(t, f) = |\text{STFT}(\text{IHR}(t))|.$$

Apply frequency mask  $B(f)$ .

### 8.4 Step 3: Train StabiFormer-RR with BL-TFOT + SCR-RR

Train on multi-view corrupted RR series; optimize OT distance in band + marker invariance.

### 8.5 Step 4: Evaluate invariance + rehab-proxy progress index

Compute marker sequences  $m_k$  over windows and report:

$$\text{Invariance} = \mathbb{E}[|m(x) - m(\tilde{x})|], \quad \text{ProgressIndex} = \text{Std}(\{m_k\}_{k=1}^K).$$

Optionally report “phase” stability across rest→task→recovery segments.

## 9 Exact Loss Formulas (clean math)

Let  $x$  denote the input RR-derived uniform series (e.g., IHR(t)), and  $\hat{y} = f_\theta(\tilde{x})$  be the model output for a corrupted view  $\tilde{x}$ . Let  $S(\cdot)$  be STFT magnitude and  $B$  be a band mask.

### 9.1 (1) Band-limited normalized TFR distributions

$$\tilde{S}_x = B \odot S(x), \quad \tilde{S}_{\hat{y}} = B \odot S(\hat{y})$$

Flatten TF bins to index  $i$  and normalize:

$$p_i = \frac{\tilde{S}_x(i)}{\sum_j \tilde{S}_x(j) + \epsilon}, \quad q_i = \frac{\tilde{S}_{\hat{y}}(i)}{\sum_j \tilde{S}_{\hat{y}}(j) + \epsilon}.$$

### 9.2 (2) BL-TFOT: Sinkhorn-regularized OT distance

Let  $u_i = (t_i, f_i)$  be TF coordinates and cost:

$$C_{ij} = \|u_i - u_j\|_2^2.$$

Entropic OT:

$$\text{OT}_\lambda(p, q) = \min_{\pi \in \Pi(p, q)} \sum_{i, j} \pi_{ij} C_{ij} + \lambda \sum_{i, j} \pi_{ij} (\log \pi_{ij} - 1),$$

then:

$$\mathcal{L}_{\text{BL-TFOT}} = \text{OT}_\lambda(p, q).$$

### 9.3 (3) Stability-Consistency Regularization for RR (SCR-RR)

Let  $m(\cdot)$  be the scalar marker head output. For two corrupted views  $\tilde{x}^{(a)}, \tilde{x}^{(b)}$ :

$$\mathcal{L}_{\text{SCR-RR}} = \left\| m(\tilde{x}^{(a)}) - m(\tilde{x}^{(b)}) \right\|_1.$$

### 9.4 (4) Optional auxiliary time loss (weak)

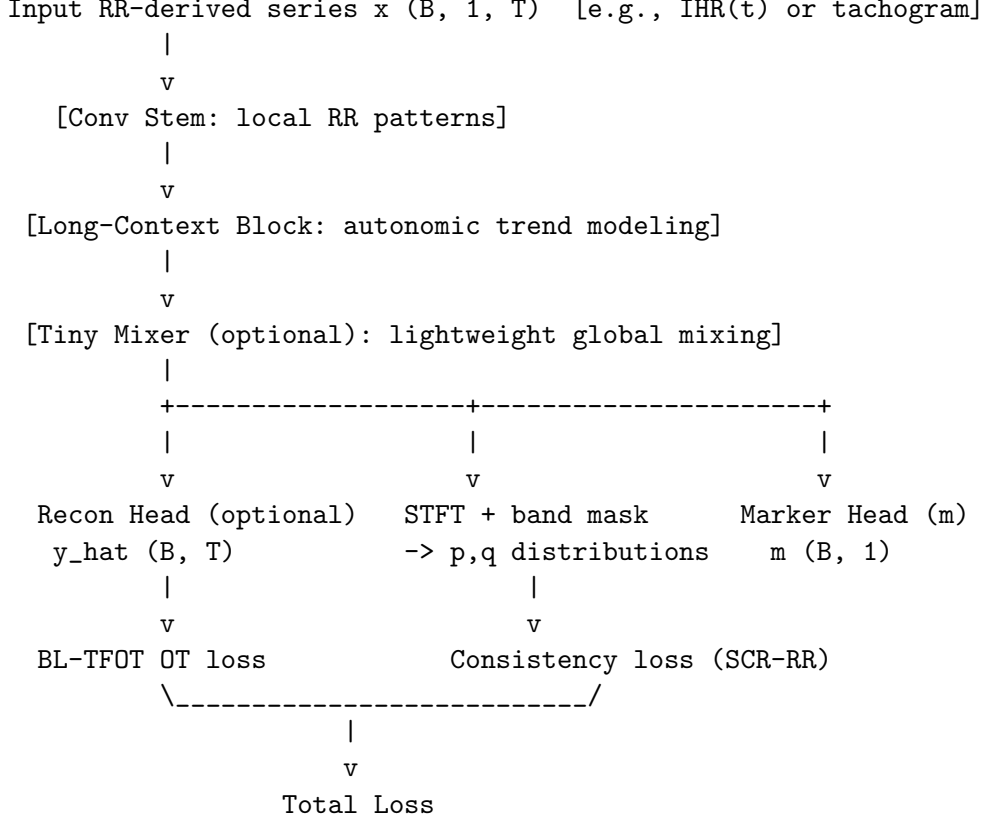
$$\mathcal{L}_{\text{MAE}} = \|x - \hat{y}\|_1.$$

## 9.5 (5) Total objective

$$\mathcal{L} = \alpha \mathcal{L}_{\text{BL-TFOT}} + \beta \mathcal{L}_{\text{SCR-RR}} + \gamma \mathcal{L}_{\text{MAE}}.$$

In artifact-heavy settings, typically  $\alpha$  and  $\beta$  dominate and  $\gamma$  is small.

## 10 StabiFormer-RR Block Diagram



## 11 Training Loop Pseudocode

---

### Algorithm 1 StabiFormer-RR Training with BL-TFOT and RR Consistency

---

- 1: Initialize model  $f_\theta$ , optimizer
  - 2: **for** each minibatch of clean RR-derived series  $x$  **do**
  - 3:   Sample  $V$  RR corruption operators  $\{c_v\}_{v=1}^V$  (missed beats, false beats, jitter, dropout)
  - 4:   Create views  $\tilde{x}^{(v)} = c_v(x)$
  - 5:   Forward:  $(\hat{y}^{(v)}, m^{(v)}) = f_\theta(\tilde{x}^{(v)})$
  - 6:   Compute STFT magnitude of  $x$  and  $\hat{y}^{(v)}$
  - 7:   Apply band mask  $B$ , normalize to distributions  $p$  and  $q^{(v)}$
  - 8:   Compute BL-TFOT:  $\mathcal{L}_{OT} = \frac{1}{V} \sum_v \text{OT}_\lambda(p, q^{(v)})$
  - 9:   Compute SCR-RR across views:  $\mathcal{L}_{SCR} = \frac{1}{V(V-1)} \sum_{a \neq b} \|m^{(a)} - m^{(b)}\|_1$
  - 10:   Optional:  $\mathcal{L}_{MAE} = \frac{1}{V} \sum_v \|\hat{y}^{(v)} - x\|_1$
  - 11:   Total:  $\mathcal{L} = \alpha \mathcal{L}_{OT} + \beta \mathcal{L}_{SCR} + \gamma \mathcal{L}_{MAE}$
  - 12:   Backprop + update  $\theta$
  - 13: **end for**
-

## 12 Minimum Viable PyTorch Code Skeleton (verbatim)

### Model + RR pipeline (skeleton)

```
# NOTE: Minimal runnable scaffold for RR-derived uniform series (e.g., IHR(t)).
# Replace ot_l2_placeholder() with Sinkhorn OT for real experiments.

import torch
import torch.nn as nn
import torch.nn.functional as F
import random

class ConvStem(nn.Module):
    def __init__(self, c_in, c_hidden=64, k=7):
        super().__init__()
        self.conv = nn.Conv1d(c_in, c_hidden, kernel_size=k, padding=k//2)
        self.bn = nn.BatchNorm1d(c_hidden)
    def forward(self, x):
        return F.gelu(self.bn(self.conv(x)))

class GatedTemporalBlock(nn.Module):
    def __init__(self, c, k=9, dilation=4):
        super().__init__()
        self.dw = nn.Conv1d(c, c, kernel_size=k,
                             padding=(k//2)*dilation, dilation=dilation, groups=c)
        self.pw = nn.Conv1d(c, 2*c, kernel_size=1)
    def forward(self, x):
        h = self.dw(x)
        a, b = self.pw(h).chunk(2, dim=1)
        return x + torch.tanh(a) * torch.sigmoid(b)

class TinyMixer(nn.Module):
    # Optional lightweight mixing; can be identity for RR.
    def __init__(self, c):
        super().__init__()
        self.proj = nn.Conv1d(c, c, kernel_size=1)
    def forward(self, x):
        return x + self.proj(x)

class StabiFormerRR(nn.Module):
    def __init__(self, c_in=1, c=64, depth=4):
        super().__init__()
        self.stem = ConvStem(c_in, c_hidden=c)
        self.blocks = nn.ModuleList([GatedTemporalBlock(c) for _ in range(depth)])
        self.mixer = TinyMixer(c)

        self.recon_head = nn.Conv1d(c, 1, kernel_size=1) # optional y_hat
        self.marker_head = nn.Sequential(
            nn.AdaptiveAvgPool1d(1),
            nn.Flatten(),
            nn.Linear(c, 1)
        )
```

```

def forward(self, x): # x: (B,1,T)
    h = self.stem(x)
    for blk in self.blocks:
        h = blk(h)
    h = self.mixer(h)
    y_hat = self.recon_head(h).squeeze(1) # (B,T)
    m = self.marker_head(h) # (B,1)
    return y_hat, m

def stft_mag(x, fs, n_fft=256, hop=64):
    # x: (B,T)
    window = torch.hann_window(n_fft, device=x.device)
    X = torch.stft(x, n_fft=n_fft, hop_length=hop, window=window,
                    return_complex=True, center=True)
    mag = torch.abs(X) # (B,F,TT)
    freqs = torch.fft.rfftfreq(n_fft, d=1.0/fs).to(x.device)
    return mag, freqs

def band_mask(freqs, fmin, fmax):
    return (freqs >= fmin) & (freqs <= fmax)

def normalize_tf(mag, mask, eps=1e-8):
    mag_b = mag[:, mask, :]
    flat = mag_b.reshape(mag_b.shape[0], -1)
    flat = flat / (flat.sum(dim=1, keepdim=True) + eps)
    return flat

def ot_l2_placeholder(p, q):
    # Placeholder (NOT true OT). Replace with Sinkhorn OT.
    return torch.mean(torch.abs(p - q))

# ----- RR corruption operators (on uniform series for simplicity) -----
def rr_corrupt_uniform(x, mode):
    # x: (B,1,T) uniform series (e.g., IHR(t))
    B, C, T = x.shape
    if mode == "dropout":
        y = x.clone()
        for b in range(B):
            s = torch.randint(0, T//2, (1,), device=x.device).item()
            e = s + torch.randint(T//10, T//4, (1,), device=x.device).item()
            y[b, :, s:e] = 0.0
        return y
    if mode == "spike":
        y = x.clone()
        idx = torch.randint(low=0, high=T, size=(B, 3), device=x.device)
        amp = 0.5 * torch.randn(B, 3, device=x.device)
        for b in range(B):
            for j in range(3):
                y[b, :, idx[b, j]] += amp[b, j]
        return y

```



```

if mode == "drift":
    t = torch.linspace(0, 1, T, device=x.device)[None, None, :]
    drift = 0.2 * torch.sin(2 * 3.14159 * 0.2 * t)
    return x + drift
if mode == "jitter":
    return x + 0.02 * torch.randn_like(x)
return x

def train_step(model, batch_x, fs, optimizer, fmin, fmax,
               alpha=1.0, beta=1.0, gamma=0.05, V=2):
    # batch_x: (B,1,T) clean uniform RR-derived series (IHR/tachogram)
    model.train()
    optimizer.zero_grad()

    modes = ["dropout", "spike", "drift", "jitter"]
    views = [rr_corrupt_uniform(batch_x, random.choice(modes)) for _ in range(V)]

    y_hats, ms = [], []
    for v in views:
        y_hat, m = model(v)
        y_hats.append(y_hat) # (B,T)
        ms.append(m)         # (B,1)

    target = batch_x[:, 0, :] # (B,T)
    mag_t, freqs = stft_mag(target, fs=fs)
    msk = band_mask(freqs, fmin, fmax)
    p = normalize_tf(mag_t, msk)

    ot_losses, mae_losses = [], []
    for y_hat in y_hats:
        mag_p, _ = stft_mag(y_hat, fs=fs)
        q = normalize_tf(mag_p, msk)
        ot_losses.append(ot_l2_placeholder(p, q))
        mae_losses.append(torch.mean(torch.abs(y_hat - target)))

    L_ot = sum(ot_losses) / len(ot_losses)
    L_mae = sum(mae_losses) / len(mae_losses)

    L_scr = 0.0
    cnt = 0
    for i in range(V):
        for j in range(i+1, V):
            L_scr = L_scr + torch.mean(torch.abs(ms[i] - ms[j]))
            cnt += 1
    if cnt > 0:
        L_scr = L_scr / cnt

    loss = alpha*L_ot + beta*L_scr + gamma*L_mae
    loss.backward()
    optimizer.step()
    return float(loss.item()), float(L_ot.item()), float(L_scr.item()), float(L_mae.item())

```

## 13 Experiment Table (Ablations) shaped like a CVPR paper

| Method                                             | BL-TFOT | SCR-RR | Multi-view | Stabi |
|----------------------------------------------------|---------|--------|------------|-------|
| A1: MAE baseline (Conv)                            |         |        |            |       |
| A2: MAE + SCR-RR                                   |         | ✓      | ✓          |       |
| A3: BL-TFOT only                                   | ✓       |        |            |       |
| A4: BL-TFOT + SCR-RR                               | ✓       | ✓      | ✓          |       |
| A5: StabiFormer-RR + MAE                           |         |        |            |       |
| A6: <b>Full: StabiFormer-RR + BL-TFOT + SCR-RR</b> | ✓       | ✓      | ✓          |       |

Table 1: **CVPR-style ablations.** Inv.↓: marker invariance error under RR corruption (lower better). Stab.↓: rehab-proxy stability index (lower better).

## 14 Future Work: Adding EEG and Multimodal Disability Monitoring

After establishing StabiFormer-RR on RR-derived dynamics, we will extend the framework in two directions:

### 14.1 F1: EEG-based cortical stability marker

We will adapt the same BL-TFOT + invariance training to EEG time–frequency maps, enabling a complementary *cortical stability* marker relevant to motor recovery and rehabilitation response. Large public clinical EEG resources such as the Temple University Hospital EEG Corpus can support methodological validation at scale [5].

### 14.2 F2: EEG–ECG coupling for disability rehabilitation

We will investigate multimodal coupling between cortical rhythms and autonomic regulation by learning a shared latent space with cross-modal consistency (e.g., marker agreement under artifacts) and modality-dropout robustness. This can yield a stronger disability-oriented monitoring tool that remains functional when one modality degrades.

## 15 References (key support)

### References

- [1] MIT-BIH Arrhythmia Database Directory (Introduction). PhysioNet. Beat annotations described as located near the R-wave peak and usable for HRV studies. Available via PhysioNet.
- [2] MIT-BIH Arrhythmia Database v1.0.0. PhysioNet database entry.
- [3] Congestive Heart Failure RR Interval Database. PhysioNet database entry.
- [4] PhysioNet HRV resources / Cardiovascular Signal Toolbox (HRV computation). PhysioNet topic page.

- [5] I. Obeid and J. Picone. The Temple University Hospital EEG Data Corpus. *Frontiers in Neuroscience*, 2016.